Labelled factor bases are generic

Coefficient-labelled factor bases are generic: a 2√N lower bound

2026-10-26 · item 3 of the 2026-10-26 triage (`docs/session-logs/2026-10-26.md`) · **class: derivation (no measurement, no leaderboard row).** Graph `bound.svg`: `examples/labelled_base_bound.rs` evaluates the closed-form bound below natively and writes `bound.jsonl`, and `plot.py` only lays those records out. Nothing is measured. PDF `report.pdf`. **Lean: the bound is proved and kernel-checked** (see "Formalization").

The idea examined

Construct the factor base from labelled points `R_i = a_i G + b_i Q`, so that a **single** decomposition of a labelled target `T = aG + bQ` over the base, `T = Σ ε_j R_{i_j}`, solves the discrete log whenever the `Q`-coefficients do not cancel. That removes the linear-algebra phase and the need for a full-rank matrix. The baseline already stops at the first dependency (`ic_framework/linalg.rs`, incremental Gauss pins the target column). What is new here is labelling the base itself.

Model

- `N`: group order. `S ⊂ G` is a structured set of size `s` whose decompositions an oracle can find (small abscissae, a coset, a tower base…).
- A labelled point is useful only inside `S`, because decompositions are over `S`. Labelled points come from draws `aG + bQ`, each at least one group operation. Unless the structure maps labels into `S` (see "Exceptions"), a draw lands in `S` with probability `s/N`. So a labelled base of `B ≤ s` points costs **at least `B·N/s`**.
- A target has on average `s^m/(m!·N)` decompositions into `m` points of `S`. The oracle pays at least one operation per target and at least one per decomposition it returns: **cost ≥ `max(1, s^m/(m!·N))` per target**.
- A decomposition is useful only if all `m` points are labelled, which happens with probability `(B/s)^m`. The expected number of useful decompositions per target is then `B^m/(m!·N)`, so **about `m!·N/B^m` targets** are needed.

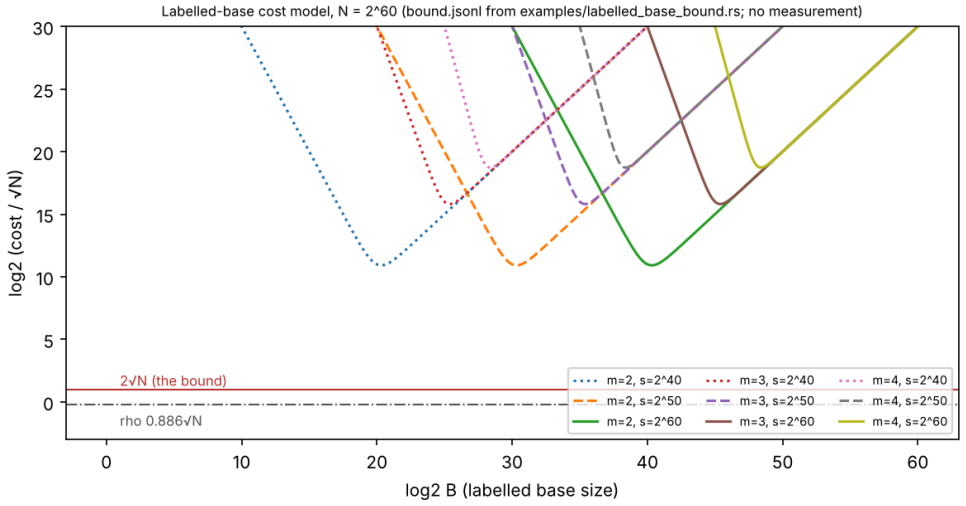
Bound

Total cost `C ≥ B·N/s + (m!·N/B^m)·max(1, s^m/(m!·N))`.

- If `s^m ≥ m!·N`: `C ≥ B·N/s + s^m/B^m ≥ 2·√(N·(s/B)^{m−1}) ≥ 2√N`, since `B ≤ s`.
- If `s^m < m!·N`: `C ≥ B·N/s + m!·N/B^m ≥ 2·√(m!·N²·B^{1−m}/s) ≥ 2·√(m!·N²/s^m) > 2√N`.

> **Any labelled-base decomposition attack costs at least `2√N` group-operation equivalents, for every structured set `S`, every base size `B ≤ s` and every `m`.** Pollard rho costs about `0.886√N` (`0.627√N` with the negation map), so the labelled base loses to rho by a factor of at least 2.3. It is a constant factor, but it holds for every structure.

The bound is loose. Evaluated at `N = 2^60` (`bound.jsonl`, `bound.svg`), the model's minimum over `B` is `2^10.92·√N` for `m = 2`, `2^15.81·√N` for `m = 3` and `2^18.72·√N` for `m = 4`, whatever the size of `S` (`log2 s` = 40, 50, 60 give the same minimum). The evaluator asserts that the minimum is at least `2√N` in every cell. Enumerating decompositions over a large `S` is the expensive part.



log2(cost/√N) of the cost model against log2 B, N = 2^60; bound.jsonl from the native evaluator

The other regime: labels without structure. A labelled base can also skip the structured oracle altogether. For example, label `B = N^{1/4}` random points, tabulate their `B²` pair sums, and test targets against the table. That is baby-step giant-step: it uses no structure of `S`, so it is a generic algorithm, and the generic-group lower bound (Shoup, about `√N`) applies; BSGS and rho attain it. Either way, a labelled base is a collision search: with structure it pays the oracle's output, and without structure it is generic.

Exceptions and what they buy

- **Automorphisms.** If `S` is closed under an automorphism group of order `k` whose action on labels is known (negation, `j = 0`/`1728`, Frobenius), one draw labels `k` points of `S`. The bound becomes about `2√(N/k)`. Rho gains the same `√k` from the same automorphisms, so the gap to rho is unchanged.
- **Labels produced by decomposition itself.** Getting labels for all of `S` without sampling is exactly what standard index calculus does: relations plus linear algebra label the base collectively. The labelled-base idea is therefore not an alternative to linear algebra. It replaces linear algebra by sampling, at cost `N/s` per label.
- **Oracles sublinear in their output.** The bound assumes the oracle pays at least one operation per decomposition it returns. An oracle that could test "is there a decomposition entirely inside `L`" for the labelled subset `L` without enumerating would

need L itself to be structured. A structured L of labelled points is the same problem one level down.

Formalization

Status: proved in Lean 4 (kernel-checked), for the inequality. The cost model is not formalized. Project lean/: LabelledBases.lean, Lean v4.34.1, Mathlib pinned at d13f23b723b8a846827a245b89c10fc7d3f11612 (the same pin as research/enge_primefield_cover_20261024/lean), lake-manifest.json committed.

report statement	Lean declaration	exact hypotheses	status
Bound, both cases (C ≥ 2√N)	LabelledBases.labelled_base_lower	N s B : ℝ, m : ℕ, 0 < N, 0 < B, B ≤ s, 1 ≤ m; conclusion 2 * √N ≤ cost N s B m, with cost N s B m = B*N/s + (m!*N/B^m) * max 1 (s^m/(m!*N))	proved
The two terms multiply to at least N	LabelledBases.terms_mul_ge	same	proved

The proof covers both cases of “Bound” at once, because it only uses that max is at least its second argument. Let x = B·N/s and y = (m!N/B^m)·max(…). Then y ≥ s^m/B^m, so x·y ≥ N·(s/B)^{m−1} ≥ N, since s/B ≥ 1. Then (x+y)² ≥ 4xy ≥ 4N and x + y ≥ 0 give x + y ≥ 2√N.

Verification, 2026-10-27: - lake exe cache get and lake build succeed with no warnings and no sorry. - #print axioms gives [propext, Classical.choice, Quot.sound] for both theorems. - The repository gate tools/lean/proof-gate.sh research/labelled_bases_20261026/lean reports PASS 2 thm 0 trusted. Its logs are in lean/receipts/. - lean4checker --fresh was not run; it is not installed here.

What the theorem does not cover: the cost model itself (the bullets under “Model”). That an attack must pay N/s per useful label, and at least one operation per returned decomposition, are modelling assumptions about attacks. They are not theorems about elliptic curves, and the Lean statement takes the cost formula as its definition.

Verdict for the triage

Item 3 is **closed by derivation** (written proof above; the inequality is proved in Lean, the cost model is an assumption). With a structured oracle, no setting of S, B or m gets a labelled-base attack below 2√N (in practice far above it). Without structure the attack is

generic and bounded by √N. The single-dependency stopping it relies on is already in the baseline. Nothing here is a measured result; the bound is exact under the stated cost model.